Progressive Reinforcement Learning from Tabular Control to Pixel-Based DQN

Comparative Analysis between different methods for levels

Course: 51.512 Reinforcement Learning for Embodied AI
Academic Year: 2025-2026
Group members: Duan Xu, Zheng Chengsheng , Manan Mehta
Submission date: 8 August 2026

Introduction-

This project studies reinforcement learning for Paddle Duel, a one-rally Pong-like control task. Rather than treating the five project levels as unrelated exercises, our work follows a progressive experimental storyline- (1) compare tabular temporal-difference control algorithms, (2) study exploration schedules against a scripted opponent, (3) test robustness across an opponent zoo, and (4) move from engineered image features to end-to-end convolutional DQN. In (5) we trained an agent using self-play and made it ready for the competition.

The experiments also exposed limitations that guide future tuning. Level 1 used an exploratory training variant that masked the movement penalty during learning; this must be reported transparently because the official environment reward includes a -0.001 movement cost. Level 3 remains weak against strong opponents, and CNN-DQN is computationally expensive and still moves frequently. These limitations are treated as findings rather than hidden.

Apart from the given prescribed evaluation metrics, other new metrics will also be introduced as we go ahead in the project and explained in detail as well as implemented.

Project Requirements and Scope-

The official project requires agents that follow the common BaseAgent interface, return one of three actions (UP, DOWN, STAY), separate training from evaluation, support save/reload without retraining, use separate training and evaluation seeds, include baselines, learning/evaluation curves, replay evidence, failure-case analysis, hyperparameter exploration, and reproducibility instructions.

Level	Official setting	Our work
1	Solo paddle practice against a wall	Q-learning vs SARSA vs Expected SARSA; diagnostics
2	Left agent vs scripted opponent	Q-learning vs SARSA; constant/linear/exponential epsilon schedules
3	Evaluation against opponent zoo	Expected SARSA trained on zoo; robustness across five opponents
4	Pixel-only observation	Feature-DQN vs CNN-DQN
5	Two learned agents	Self-play

Environment, State, Actions, and Rewards-

Each episode begins with the ball at the centre and ends when either side scores or the 60 s limit is reached (30 fps, maximum 1800 environment steps). The learning agent controls the left paddle. Actions are 0=UP, 1=DOWN, and 2=STAY.

Event	Official reward
UP or DOWN	-0.001
STAY	0
Successful paddle hit (first 10)	+0.06
Successful paddle hit (after 10)	+0.02
Left scores	+10 to Left; episode ends
Right scores	+10 to Right; episode ends
Timeout	No scoring bonus; cumulative reward decides winner

For Levels 1-3 the environment supplies a seven-element tabular state: the controlled paddle y-bin, opponent paddle y-bin, ball x-bin, ball y-bin, horizontal and vertical velocity signs, and speed bin. Level 4 instead supplies a 150x320x3 uint8 RGB image.

Experimental Methodology-

The experiments use controlled comparisons wherever possible. Training uses seed 0 with episode-dependent offsets, while evaluation uses unseen seeds beginning at 10,000. Final evaluations typically use 200 episodes. Agents are evaluated greedily with exploration disabled. In addition to reward and win rate, we record paddle hits, episode length, movement/stay counts, miss rate, timeout rate, and replay trajectories. Saved Q-tables or PyTorch checkpoints are reloaded and evaluated to verify reproducibility.

Across the tabular experiments, the common starting values were alpha=0.1 and gamma=0.99. Most final notebooks used 2,000 training episodes with the first 400 episodes reserved for forced random exploration. The neural experiments also used 2,000 episodes.

Level 1 - Tabular Control Algorithm Comparison

Goal-

The first research question was: which classical tabular TD-control method provides the best behaviour in the simplest setting? We implemented Q-learning (off-policy), SARSA (on-policy), and Expected SARSA (expected on-policy backup) using a shared agent interface.

Training design-

The final diagnostic notebook used 2,000 episodes, 400 forced-random episodes, alpha=0.1, gamma=0.99, epsilon_start=1.0, epsilon_min=0.05, and epsilon_decay=0.9999. During this exploratory Level 1 run, the movement penalty was masked in the learning update to encourage the agent to pursue hit rewards instead of prematurely preferring STAY. Evaluation still used the official environment rewards. This is a deliberate training modification and should be discussed with the instructor before treating it as the final official Level 1 configuration.

Results-

Agent	Mean reward	Mean hits	Mean steps	Move actions	Learned states
Q-learning	-0.0445	2.57	437.0	272.18	33,082
SARSA	-0.0864	1.81	336.37	222.69	32,872
Expected SARSA	-0.1202	2.72	465.70	296.21	32,336

All three agents remained weak under the official scoring objective in this Level 1 configuration, but their behavioural profiles differed. Expected SARSA produced the most hits and longest episodes, Q-learning achieved the least-negative mean reward, and SARSA moved less. The replay diagnostics confirmed that the paddle was genuinely moving rather than remaining fixed.

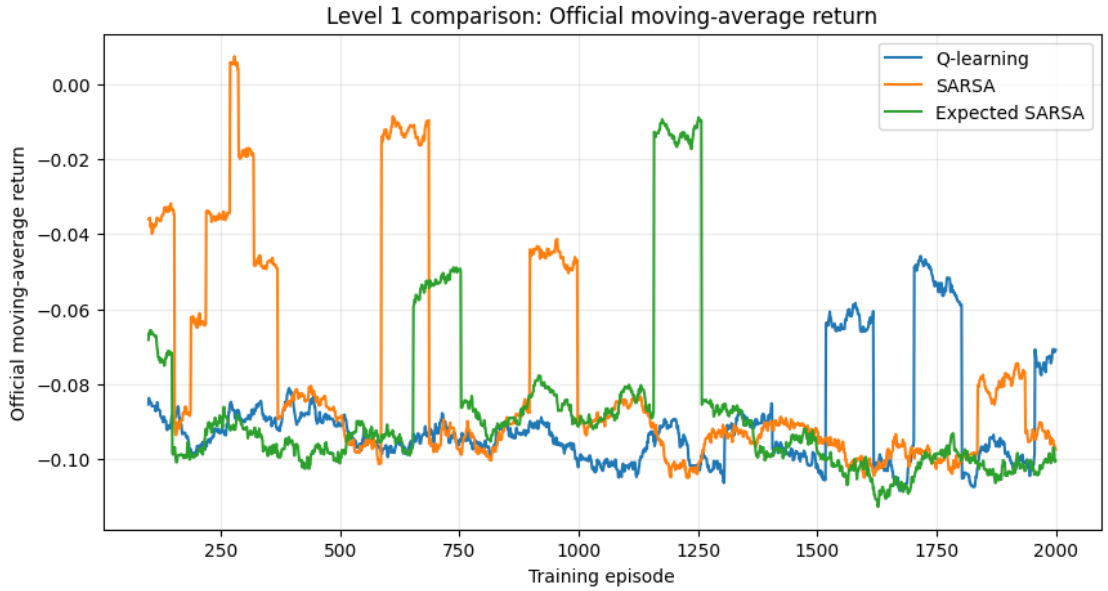

Figure 1. Representative Level 1 training comparison curve.

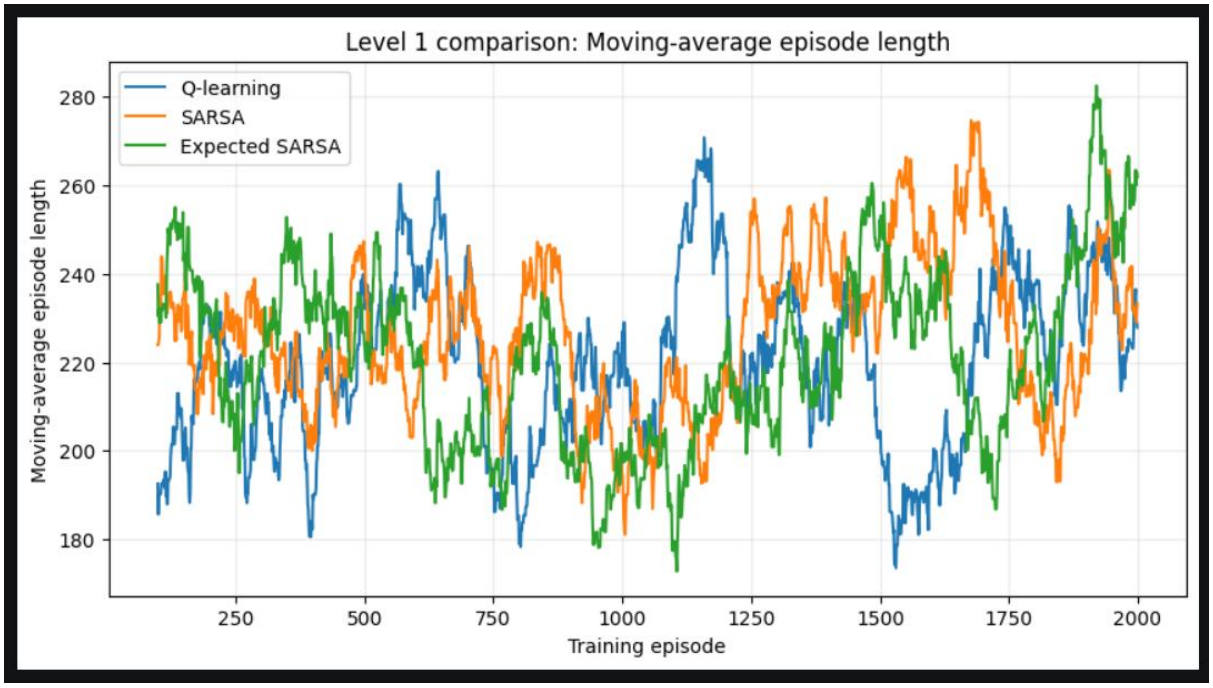

Figure 2. Representative Level 1 episode comparison curve.

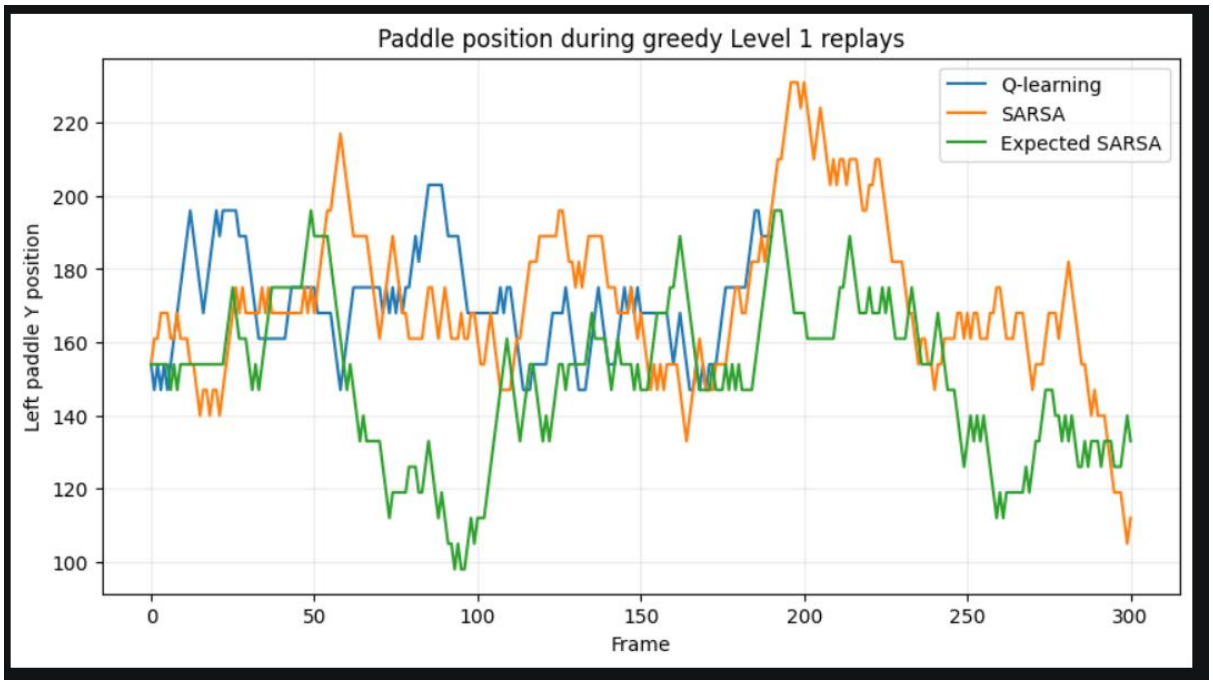

Figure 3. Paddle position during Level 1 replay

Level 2 - Algorithm and Exploration Studies

Q-learning versus SARSA-

Level 2 introduces a scripted opponent. We first compared Q-learning and SARSA under the same 2,000-episode training budget and 400-episode forced-random phase. This isolates the effect of the TD target before changing the exploration strategy.

Agent	Mean reward	Win rate	Mean hits	Mean steps	Miss rate
Q-learning	2.1649	21.5%	1.11	191.50	78.5%
SARSA	1.8624	18.5%	0.94	170.83	81.5%
Random baseline	1.5015	15.5%	0.77	141.75	84.5%
Tracking baseline	7.6266	75.5%	7.98	741.79	24.0%

Q-learning was the stronger learned tabular agent on the principal metrics, exceeding SARSA and the random baseline. However, both remained far below the tracking benchmark, showing that the discretized state and limited training budget still leave substantial room for improvement.

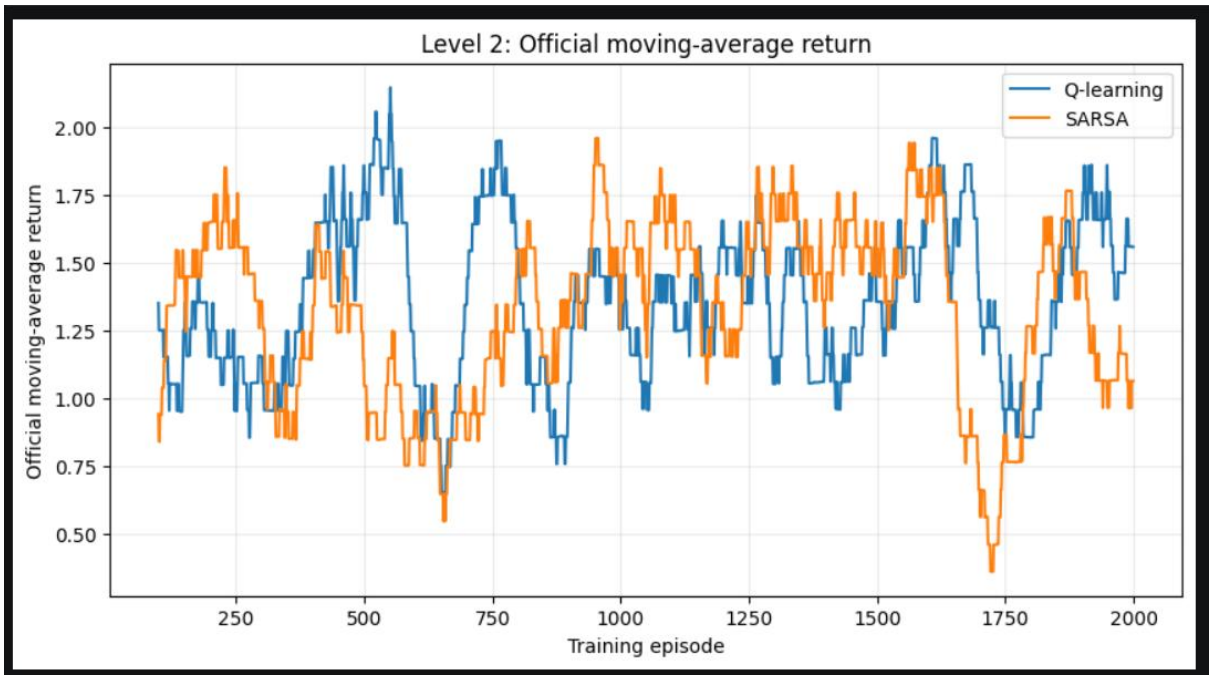

Figure 4. Moving-average return during Level 2 replay

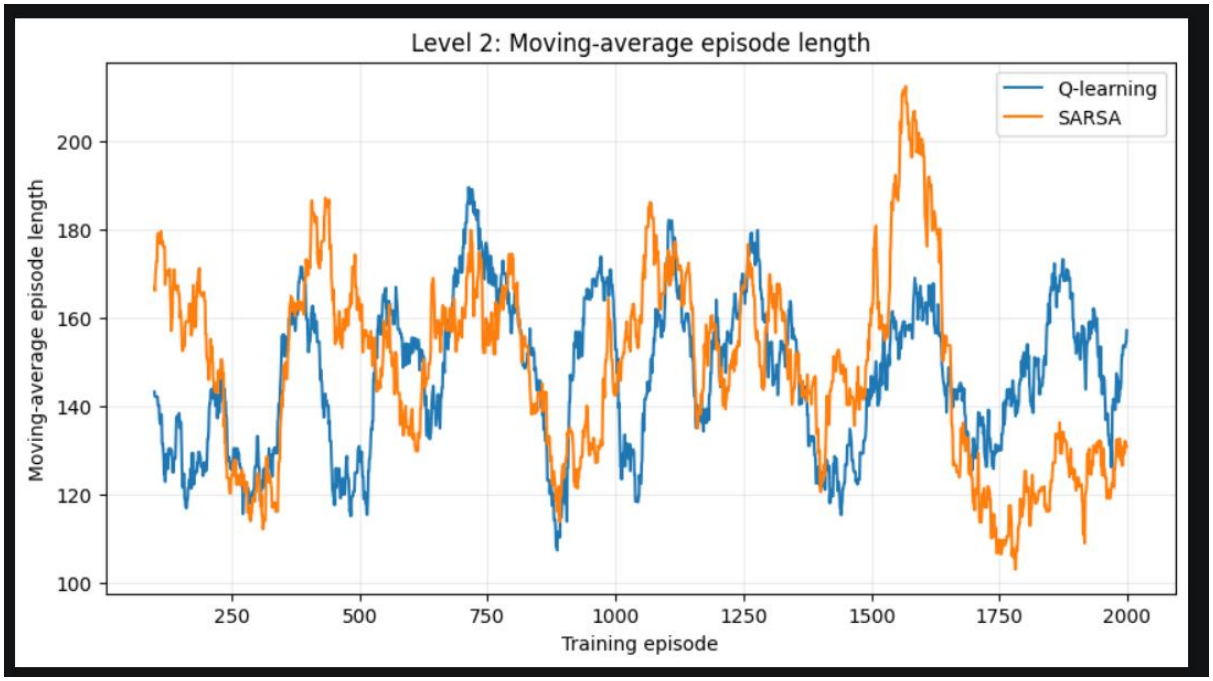

Figure 5. Moving-average episode length during Level 2 replay

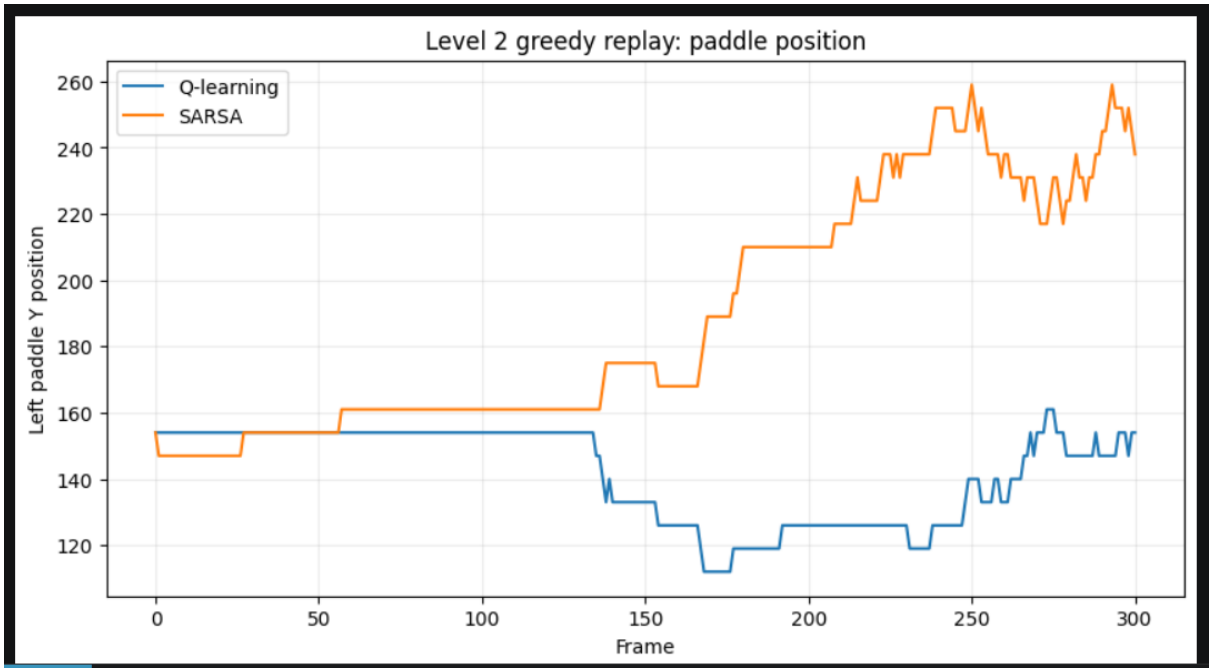

Figure 6. Paddle position during Level 2 replay

Exploration schedule comparison-

We then fixed the algorithm to Q-learning and compared three epsilon schedules after the same 400-episode forced-random phase: constant epsilon=0.20, linear decay from 1.00 to 0.05, and exponential decay from 1.00 to 0.05.

Schedule	Mean reward	Win rate	Mean hits	Mean steps	Move actions
Constant	1.6729	16.5%	1.07	191.07	41.32
Linear	2.0746	20.5%	1.245	224.62	50.14
Exponential	1.8778	18.5%	1.025	186.89	33.75

The linear schedule gave the best aggregate evaluation performance, suggesting that gradual exploration reduction offered a better exploration-exploitation balance than either persistent exploration or the tested exponential decay.

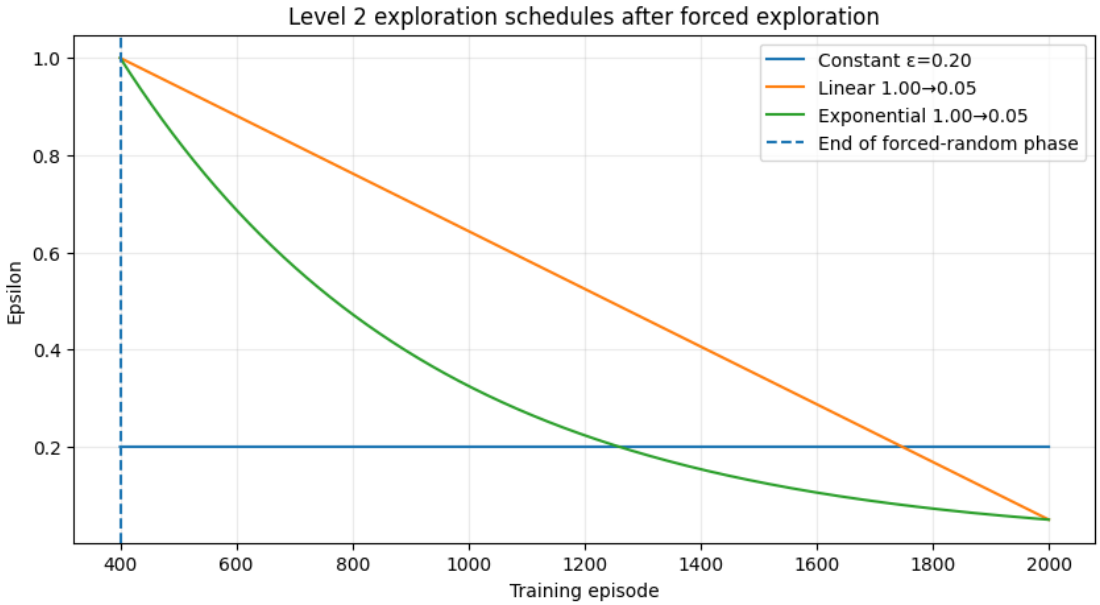

Figure 7. Epsilon schedules used in the Level 2 exploration experiment.

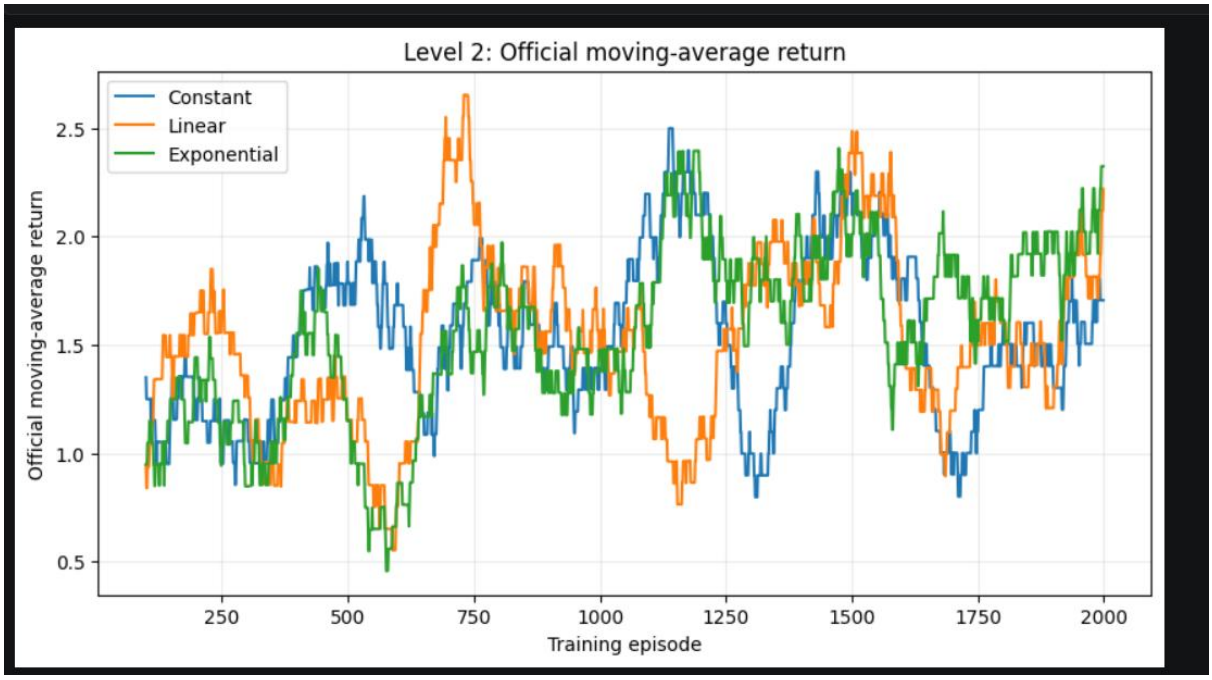

Figure 8. Moving-Average return for Level 2 exploration experiment.

Level 3 - Expected SARSA Robustness

Motivation and design

The Level 3 question was not simply whether an agent could beat one opponent, but whether a policy trained against a mixed opponent distribution would generalize. Expected SARSA was trained for 2,000 episodes against the built-in 'zoo' opponent using alpha=0.1, gamma=0.99, epsilon_start=1.0, epsilon_min=0.05, epsilon_decay=0.9999, and 400 forced-random episodes. The resulting Q-table contained 77,102 states.

Robustness results

Opponent	Mean reward	Win rate	Mean hits	Mean steps	Miss rate
Random	2.4932	25.0%	0.60	114.99	75.0%
Weak	1.5004	15.0%	0.88	163.13	85.0%
Tracking	0.8090	8.0%	1.13	195.68	92.0%
Strong	0.6597	6.5%	1.20	206.92	93.5%
Zoo	1.7062	17.0%	0.87	158.07	83.0%

The average win rate across opponents was 14.3%, with a worst-case win rate of 6.5%. The strong opponent was the hardest by both win rate and reward. Interestingly, stronger opponents produced more hits and longer episodes because they returned the ball more consistently, but they still won more often. This distinction shows why win rate, reward, hits, and survival time should be interpreted together.

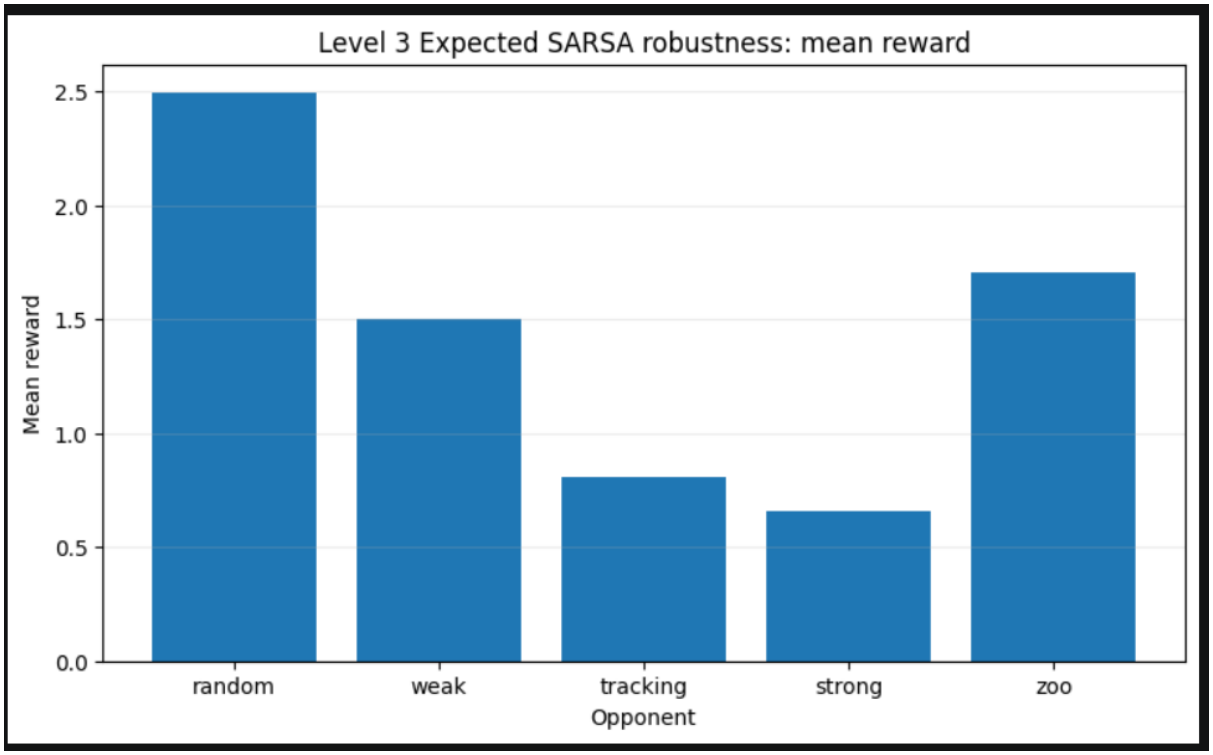

Figure 9. Level 3 mean reward across opponent types

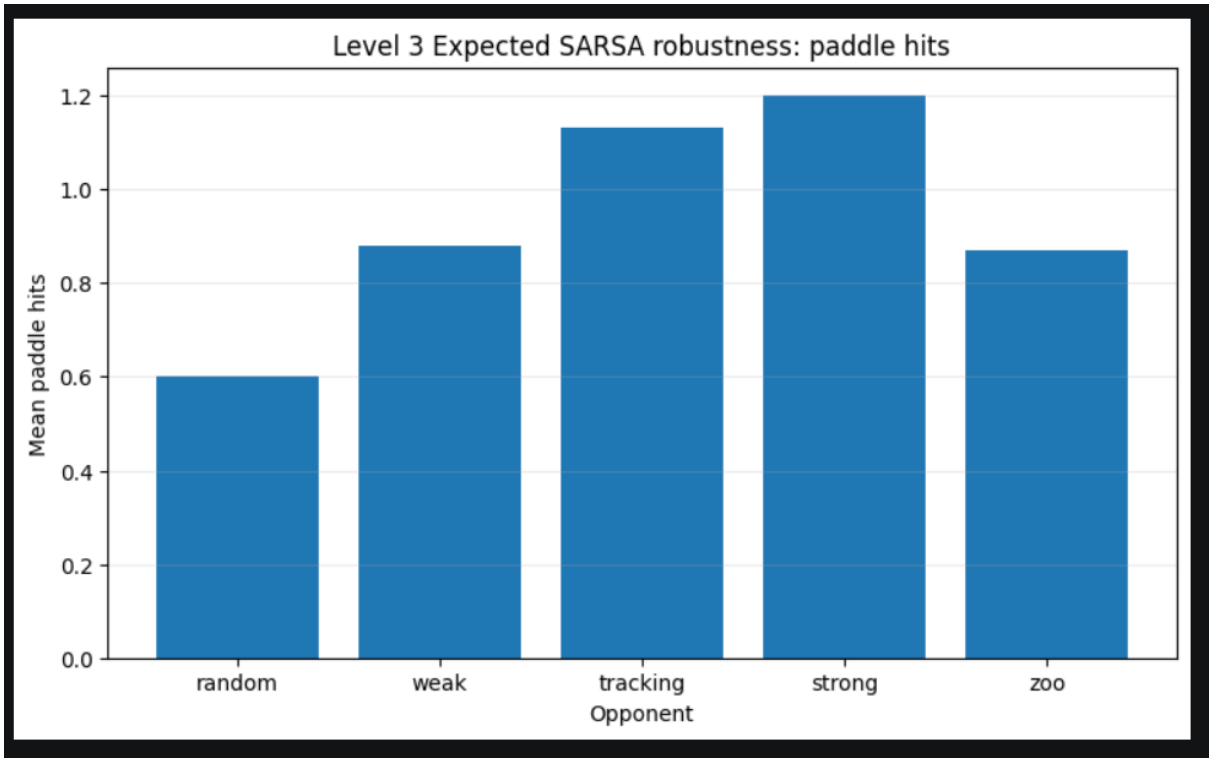

Figure 10. Level 3 mean paddle hits across opponent types

This demonstrated the limitations of tabular RL and motivated moving toward function approximation.

Level 4 - From Engineered Features to End-to-End Vision

Level 4 changes the observation from a compact tabular state to pixels. We deliberately used two DQN variants to separate the value of function approximation from the value of learned visual representation.

Feature-DQN

The first neural baseline extracted six normalized coordinates from each RGB frame (left paddle x/y, right paddle x/y, ball x/y) and concatenated their frame-to-frame differences, producing a 12-dimensional state. An MLP with two 128-unit ReLU layers mapped this vector to three Q-values. The agent used experience replay, a target network, Adam, Huber loss, and gradient clipping.

Metric	Feature-DQN	Random	Tracking
Mean reward	1.9836	1.5015	7.6266
Win rate	20.5%	15.5%	75.5%
Mean steps	172.23	141.75	741.79
Mean hits	1.075	0.77	7.98

Feature-DQN improved over random play but remained close to the tabular Level 2 performance. Its replay also showed excessive motion (107 UP, 24 DOWN, only 16 STAY actions in the selected episode), suggesting that the engineered feature representation and learned policy were still limited.

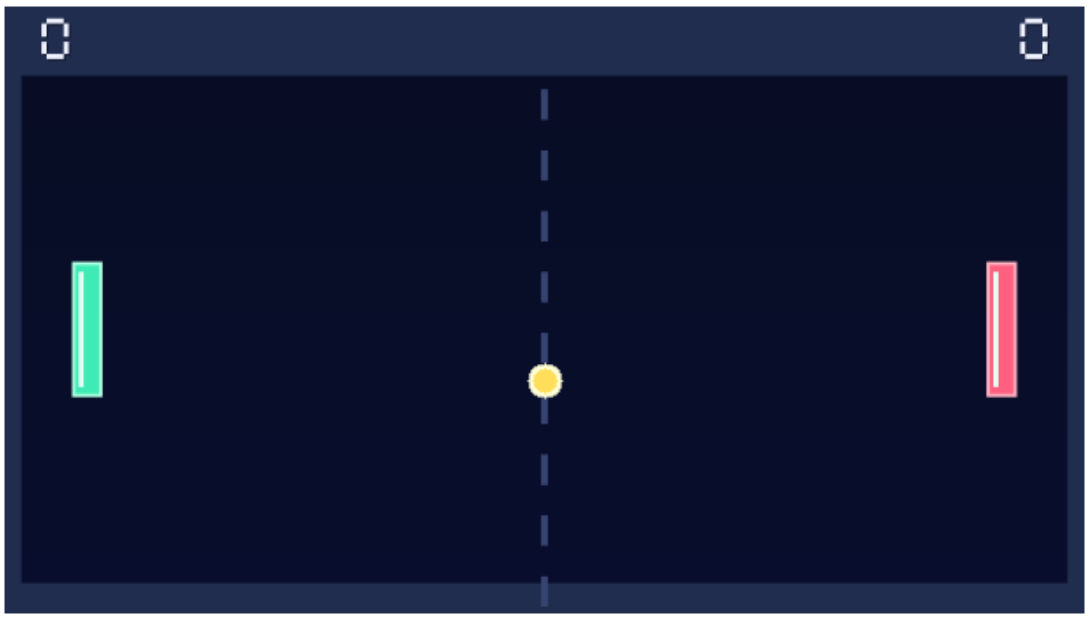

Figure 11. Level 4 RGB observation and extracted-object setting used by Feature-DQN.

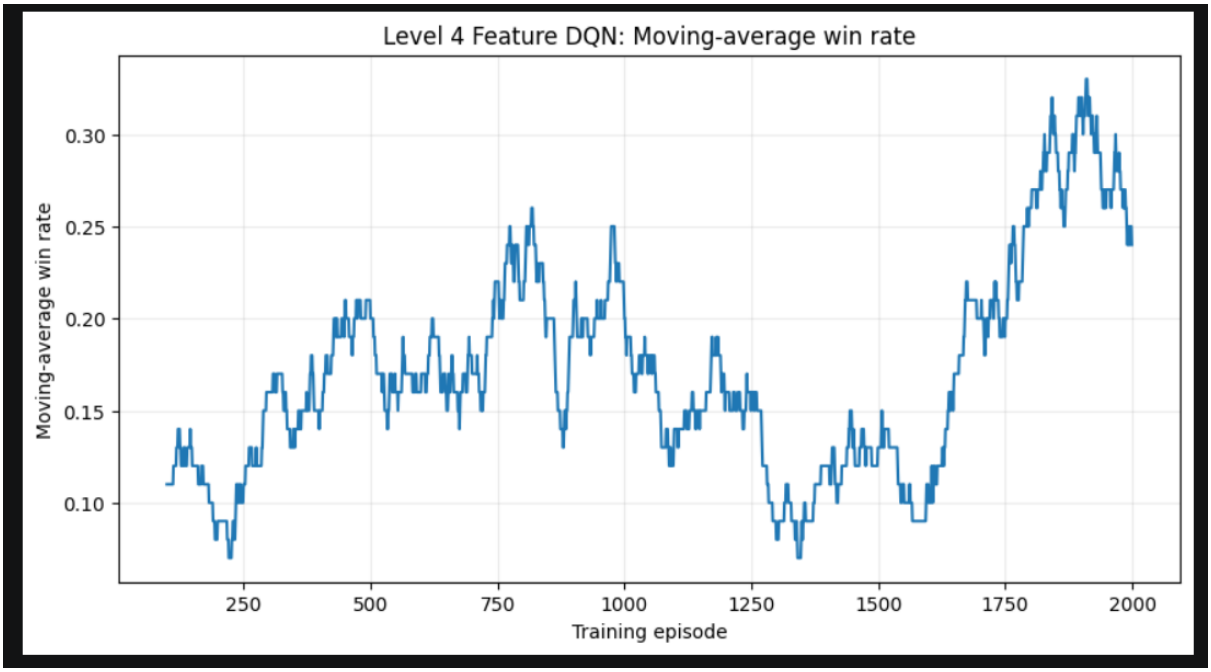

Figure 12. Representative Feature-DQN training curve.

CNN-DQN

The second agent removed explicit coordinate extraction. Each RGB frame was converted to grayscale, resized to 84x84, and four consecutive frames were stacked to provide temporal information. The CNN used three convolutional layers (32 filters 8x8/stride 4; 64 filters 4x4/stride 2; 64 filters 3x3/stride 1), followed by a 512-unit fully connected layer and three Q-value outputs.

Training used a 30,000-transition uint8 replay buffer, batch size 64, learning rate 1e-4, gamma=0.99, epsilon_start=1.0, epsilon_min=0.05, epsilon_decay=0.99995, a 5,000-transition warm-up, target updates every 2,000 environment steps, and optimization every four steps. Training ran on an NVIDIA GeForce RTX 5050 Laptop GPU.

Metric	CNN-DQN	Feature-DQN	Random	Tracking
Mean reward	6.9040	1.9836	1.5015	7.6266
Win rate	70.0%	20.5%	15.5%	75.5%
Mean hits	3.67	1.075	0.77	7.98
Mean steps	482.36	172.23	141.75	741.79

CNN-DQN was the clearest improvement in the project. Its 70.0% evaluation win rate was 49.5 percentage points above Feature-DQN and only 5.5 points below the tracking benchmark. Reloading the saved checkpoint produced a 72.0% win rate on a separate 50-episode evaluation, confirming that the trained model could be restored without retraining.

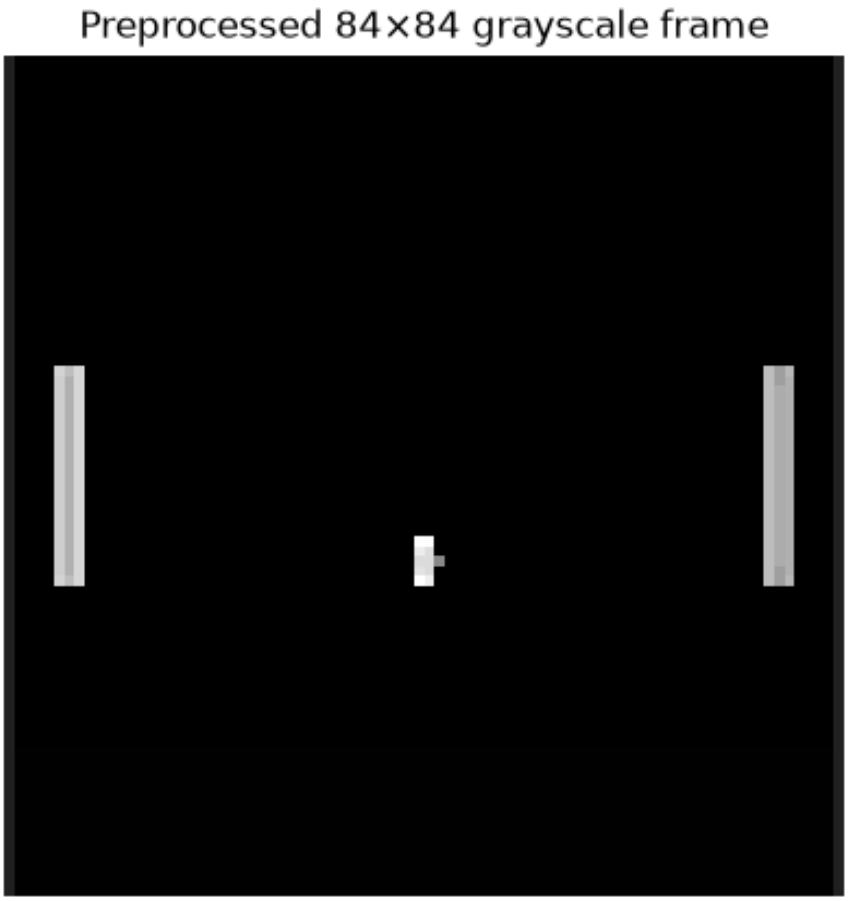

Figure 12. CNN-DQN preprocessing: grayscale 84x84 frame used in a four-frame stack.

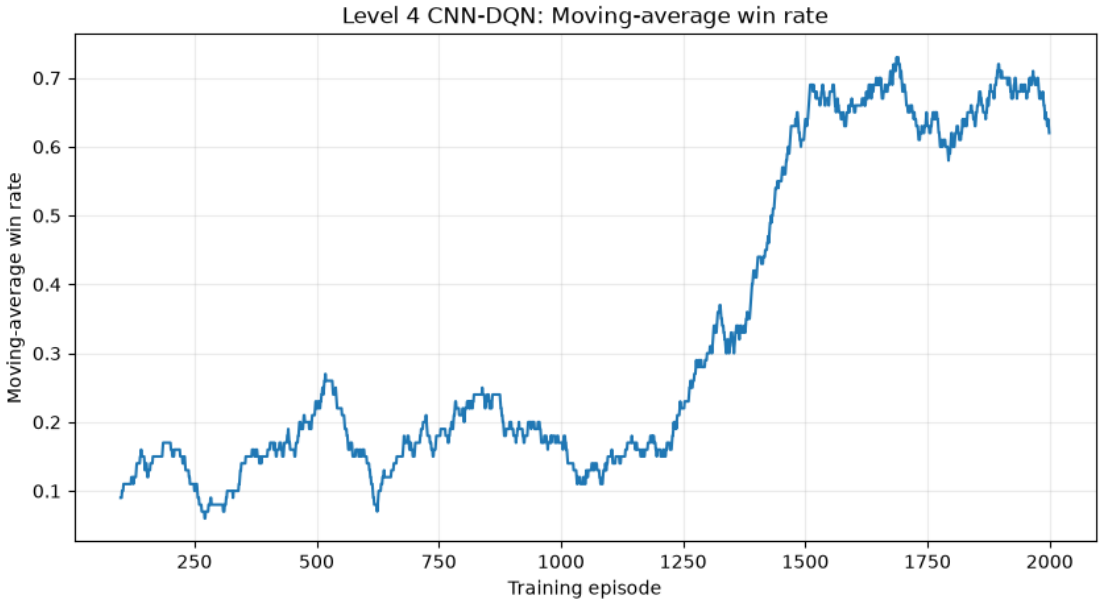

Figure 13. Representative CNN-DQN training curve.

Cross-Level Discussion

The experiments form a progression in both algorithmic complexity and representation learning. At Levels 1-3, the environment supplies a compact discretized state, so performance is primarily determined by the TD update rule, exploration, and coverage of the Q-table. At Level 4, the observation becomes visual. Feature-DQN manually reconstructs a compact state from pixels, whereas CNN-DQN learns spatial features directly and uses frame stacking to infer motion. The large CNN-DQN improvement indicates that representation quality was a major bottleneck in the feature-based approach.

Experiment	Primary question	Main finding
Level 1	Which tabular TD method behaves best?	Algorithms show different reward/hit/movement profiles; no decisive strong scorer yet.
Level 2A	Q-learning or SARSA vs scripted opponent?	Q-learning: 21.5% vs SARSA: 18.5% win rate.
Level 2B	Which epsilon schedule?	Linear decay led the tested schedules at 20.5% win rate.
Level 3	Does Expected SARSA generalize?	Generalizes to all tested opponents but strong/tracking expose a large robustness gap.
Level 4	Do learned visual features help?	CNN-DQN reached 70%, far above Feature-DQN at 20.5%.

Failure Cases and Limitations

- Level 1 reward handling: the final diagnostic experiment masks the movement penalty during training. This helped prevent early collapse to STAY, but it is not identical to learning directly from the official reward. The final submission should either justify this as reward shaping or rerun a strictly official-reward baseline.
- Tabular sparsity and coverage: tens of thousands of discrete states were learned, yet many state-action values remain weakly estimated. This contributes to low win rates and sensitivity to opponent behaviour.
- Level 3 robustness: the strong opponent reduced win rate to 6.5%. Longer rallies against strong opponents should not be mistaken for better performance; the agent survives longer but still loses more often.
- Feature-DQN representation: coordinate extraction is compact and interpretable but discards visual context and may provide only a crude estimate of motion.
- CNN-DQN cost: training required about 500,379 environment steps and 123,845 optimization steps. It is substantially more expensive than the tabular methods.
- CNN-DQN movement efficiency: the trained agent averaged about 316 movement actions per evaluation episode versus 166 STAY actions. Future work could examine anti-chattering or movement-aware tuning without compromising win rate.
- Fairness of neural comparison: both neural agents used 2,000 episodes, but CNN-DQN accumulated more environment steps because its episodes lasted longer. A stricter comparison could equalize total environment interactions.

Hyperparameter Tuning Plan

The project instructions explicitly request hyperparameter exploration. The current notebooks already compare exploration schedules and contain several controlled parameter choices, but the final version can be strengthened with a compact tuning stage rather than introducing many new algorithms.

Level	Recommended final tuning
1	Compare official reward vs masked movement penalty; alpha and epsilon-decay sensitivity.
2	Retain schedule comparison; optionally test alpha in {0.05, 0.1, 0.2} for the best schedule.
3	Compare zoo-only training with balanced and hard-opponent-weighted mixtures; optimize worst-case win rate.
4	Tune learning rate, target-update interval, replay size, and epsilon decay; optionally compare 2-frame vs 4-frame stacking.

Conclusion

The project demonstrates a deliberate progression from tabular reinforcement learning to deep visual control. The tabular experiments established algorithmic and exploration baselines, while the robustness study showed that success against one opponent does not imply generalization. The Level 4 comparison produced the strongest result: CNN-DQN substantially outperformed both Feature-DQN and the random baseline and approached the handcrafted tracking controller. The next priority is not to add many unrelated methods, but to perform targeted hyperparameter tuning, improve worst-case robustness, and ensure that all final comparisons use clearly documented and defensible reward/training settings.

Appendix- Saved Agents

File	Purpose
q_learning.pkl / sarsa.pkl / expected_sarsa.pkl	Level 1 tabular agents
q_learning_level2.pkl / sarsa_level2.pkl	Level 2 algorithm comparison
q_learning_constant_level2.pkl	Level 2 constant-epsilon Q-learning
q_learning_linear_level2.pkl	Level 2 linear-decay Q-learning
q_learning_exponential_level2.pkl	Level 2 exponential-decay Q-learning
expected_sarsa_level3_zoo.pkl	Level 3 robustness agent
feature_dqn_level4.pt	Level 4 Feature-DQN
cnn_dqn_level4.pt	Level 4 CNN-DQN